Autonomous AI Tutor Assistant

Advanced AI Engineering Project (3 Students, 6 Weeks)

Project Theme

Building a Proactive Multi-Agent AI System for Tutor Teachers

---

1. Introduction

This project is intentionally designed as an **AI engineering learning project**, not merely a software development project.

The goal is not to build the simplest possible solution.

The goal is to learn:

- AI agents
- MCP (Model Context Protocol)
- LangGraph
- RAG systems
- AI tool usage
- AI orchestration
- agent autonomy
- modern AI architecture patterns

through hands-on experimentation.

Students are encouraged to investigate different approaches, compare technologies, build prototypes, document findings, and ultimately create a functioning proof-of-concept system.

---

2. Project Vision

Imagine a tutor teacher opening Telegram on Monday morning and seeing:

> Good morning.
>
> Based on this week's tutoring calendar:

🔴 Two students appear to be significantly behind expected progress.

🟡 One student's study right expires within 12 months.

🟢 First-year student check-in discussions should be organized this month.

Would you like details?

The teacher did not ask for this information.

The system discovered it autonomously.

That is the core idea of this project.

---

3. Learning Goals

After completing this project students should understand:

AI Systems

- What is an AI Agent?
- What is an MCP server?
- What is LangGraph?
- What is RAG?
- What is an autonomous workflow?

Software Engineering

- Modern AI architecture
- API integrations
- Database modelling
- Event-driven systems
- Scheduling and automation

Research Skills

- Reading technical documentation
- Comparing alternative frameworks
- Evaluating tradeoffs
- Reporting findings

---

4. Project Philosophy

Students are expected to spend significant time investigating technologies.

A successful project is not necessarily the one with the most features.

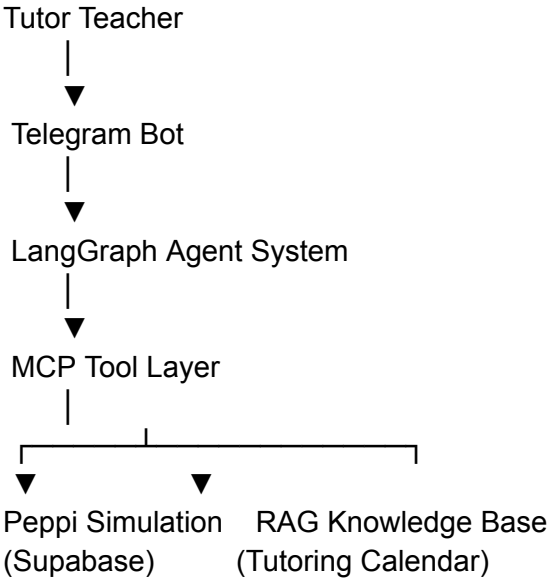
A successful project is one where students can explain:

- Why they chose a solution
- Why they rejected alternatives
- What worked
- What did not work

---

5. System Overview

The proposed system consists of five major parts.



---

6. Simulated Peppi Environment

The project must simulate a simplified Peppi system.

No real student information should be used.

The purpose is to create realistic test data.

Student Table

Student

Student Number
Name
Group
Programme
Start Date
Status

Study Rights Table

This is one of the most important parts of the project.

Study Right

Student
Start Date
End Date
Status

Extension Count

Examples:

ACTIVE

EXPIRES_SOON

EXTENDED

GRADUATED

Course Completions

Course Completion

Student
Course
Credits
Grade
Date

Curriculum Requirements

The curriculum table describes expected progression.

Example:

Semester 1
30 ECTS

Semester 2
60 ECTS

Semester 3
90 ECTS

---

7. Why Study Rights Matter

Most student analytics systems focus only on credits.

This project should go further.

Example:

Student A:

Study right ends:
2027-05-01

Credits:
100 / 240

Potential concern.

Student B:

Study right ends:
2027-05-01

Credits:
235 / 240

Probably not a concern.

The AI should learn to distinguish between these situations.

---

8. Tutor Calendar as the Heart of the System

The uploaded tutoring calendar should become the primary knowledge source.

Students should investigate:

What is RAG?

Why use RAG?

How should documents be chunked?

Which information should become embeddings?

Suggested Investigation Task

Each student should read about:

- vector databases
- embeddings
- semantic search

and explain them to the team.

---

9. Agent Architecture

The system should not be built as a single chatbot.

The recommended architecture uses specialized agents.

Calendar Agent

Purpose:

Understand the tutoring calendar.

Questions:

- What activities should happen this week?
- What actions are relevant now?

Progress Analysis Agent

Purpose:

Analyze student progression.

Questions:

- Is progression normal?
- Is the student behind expectations?

Study Right Agent

Purpose:

Monitor study rights.

Questions:

- Is a study right expiring?
- Is intervention needed?

Recommendation Agent

Purpose:

Generate tutor recommendations.

Example:

> Schedule a discussion with the student.

Communication Agent

Purpose:

Create messages and summaries.

Example:

> Draft an email or Telegram message.

---

10. Why LangGraph?

Students should investigate:

Traditional Chatbot

Question
↓
LLM
↓
Answer

Agent Workflow

Question
↓
Planner
↓
Tools
↓
Analysis
↓
Response

LangGraph helps model these workflows.

---

11. Why MCP?

Students should investigate MCP carefully.

This project is an excellent opportunity to learn it.

Instead of:

Agent
↓
Database

Use:

Agent
↓
MCP Tool
↓
Database

Example tools:

get_student()

get_progress()

get_study_right()

find_students_at_risk()

---

12. Suggested Student Responsibilities

With only three students, I would divide the work like this.

Student 1

Data & Simulation Lead

Responsible for:

- Supabase
- Peppi simulation
- Study rights
- Curriculum data

Research topics:

- PostgreSQL
- Supabase
- Data modelling

Student 2

Knowledge & AI Lead

Responsible for:

- RAG
- Tutoring calendar
- Embeddings
- Vector search

Research topics:

- RAG
- Chunking
- Vector databases
- Retrieval quality

Student 3

Agent & Integration Lead

Responsible for:

- LangGraph
- MCP
- Telegram
- Scheduler

Research topics:

- LangGraph
- MCP
- Telegram bots
- Agent orchestration

---

13. Research Phase (Week 1)

The first week should focus almost entirely on research.

Each student should produce:

Technology Summary

1–2 pages explaining:

- what the technology is
- why it exists
- why it might help this project

Required Topics

Student 1

Research:

- Supabase
- PostgreSQL
- Database modelling

Student 2

Research:

- RAG
- Embeddings
- Vector databases

Student 3

Research:

- LangGraph
- MCP
- Telegram bots

---

14. Prototype Phase (Week 2)

Build small experiments.

Not the final system.

Examples:

Prototype 1

Telegram bot replies:

Hello

Prototype 2

RAG answers questions about the tutoring calendar.

Prototype 3

Agent retrieves a student from Supabase.

Goal:

Learn before integrating.

---

15. Integration Phase (Weeks 3–4)

Combine the prototypes.

Target architecture:

Telegram
↓
LangGraph
↓
MCP
↓
Supabase

and

RAG

---

16. Autonomous Workflow Phase (Week 5)

This is where the project becomes interesting.

Students should build:

Scheduler

Example:

Every Monday
08:00

Workflow:

Check tutoring calendar
↓
Check students
↓
Check study rights
↓
Generate recommendations
↓
Send Telegram notification

---

17. Demonstration Scenario

A successful demo could look like this:

Monday Morning

System runs automatically.

Finds:

Student A

Behind by 25 ECTS

Student B
Study right expires in 9 months

Telegram message:

> Good morning.
>
> Two students require attention.
>
> Would you like details?

Teacher:

details

Agent:

Provides explanation.

---

18. Stretch Goals

Only if time allows.

Hermes

Research:

Can Hermes replace LangGraph?

OpenClaw

Research:

Could OpenClaw support autonomous educational agents?

Multi-Agent Collaboration

Experiment with agent-to-agent communication.

---

19. Evaluation Criteria

25%

Research quality

20%

Architecture quality

15%

Technical implementation

15%

Documentation quality

10%

Prototype quality

10%

Demonstration

5%

Innovation

---

20. Final Advice to the Team

Do not start by coding the final system.

Start by understanding the technologies.

A recommended order is:

Step 1

Learn Supabase

Step 2

Learn RAG

Step 3

Learn LangGraph

Step 4

Learn MCP

Step 5

Build small prototypes

Step 6

Integrate everything

Step 7

Add autonomous scheduling

Step 8

Prepare a convincing demonstration

If the team follows this approach, they will learn substantially more about modern AI engineering than by immediately jumping into implementation. This is exactly the kind of project that exposes students to the architectural patterns currently used in advanced AI products and autonomous agent systems.